Image Project

Foundamentals of Sound and Image

Authors:
Andrea Rodríguez Rodríguez
Diego Martínez Graña
Carlota Iglesias Martínez
Lucía Castrillo Sánchez

Tasks:
Task 4
Task 3
Task 2 and 5
Task 1, 6 and LaTeX report

Abstract

Optical Character Recognition (OCR) has been a fundamental technology for document digitization, information accessibility, and archival systems since the mid-20th century, as it converts printed or handwritten text in images into machine-readable text. Despite the rise of deep learning approaches, classical image processing methods remain relevant for controlled-domain applications where documents exhibit consistent structure, fixed fonts, and manageable lighting conditions. This project implements a complete end-to-end OCR pipeline for capital-letter documents using sequential image processing techniques in MATLAB.

The system architecture comprises six integrated stages: binarization (via Otsu's adaptive threshold method), vertical projection-based row detection and segmentation, horizontal projection-based character boundary detection, geometric normalization to fixed-size images, alphabet creation through reference document processing, and finally template-based character recognition using normalized 2D correlation.

Contents

1 Introduction

1.1 Project constraints and scope

This project focuses on a constrained but realistic scenario: documents containing only capital letters in Arial font, organized in horizontal rows, and scanned at reasonable resolution.

Despite the given constraints making the creation of the project easier, the implementation of the full pipeline from raw image input to final text output makes the project ambitious in breadth. This end-to-end approach reveals how errors at one stage propagate to downstream stages, a critical insight that single-stage misses.

Additionally, the project explores alternative methods at each stage: both base implementations (simpler, faster) and advanced variants (more robust, slower), both recommended by the guide and others self-discovered. At the end, informed trade-off decisions could be taken based on measured performance and will be later discussed.

1.2 Problem definition and success criteria

The overarching goal of this project is to build a functional OCR system that accurately recognizes capital letters in clean and moderately degraded documents, while thoroughly understanding and documenting the performance characteristics, failure modes, and improvement pathways of each component. This goal naturally decomposes into several specific objectives:

1. Implement a complete OCR pipeline that processes raw scanned images and outputs recognized text.
2. Achieve high recognition accuracy (targeting above 95%) on a test set of documents with varying levels of degradation.
3. Analyze the performance of each stage (binarization, segmentation, normalization, recognition) to identify bottlenecks and error sources.
4. Explore and compare alternative methods for each stage to understand their trade-offs in terms of accuracy, robustness, and computational efficiency.
5. Document the implementation details, results, and insights gained throughout the project in a comprehensive report.
6. Identify potential improvements and future work.

2 Methodology

2.1 System architecture and pipeline overview

The OCR system is organized as a linear pipeline designed with modularity as a core principle. Each of the six stages has a well-defined interface: it accepts input data, processes it deterministically, and returns output that feeds into other stage. This sequential organization reflects the logical flow of the OCR problem: we must first separate text from background (binarization), then locate where text regions are (row segmentation), then locate individual characters within those regions (character segmentation), then normalize those characters to a standard form (preprocessing), then establish what each character looks like (alphabet creation), and finally match observed characters against known templates (recognition).
The six stages are implemented as separate MATLAB functions, each with a clear purpose.

- Task 1 (`task1BinarizationAdvanced.m`) converts grayscale or RGB images into binary matrices.
- Task 2 (`task2ExtractRows.m`, `task2RowProjection.m`, `task2SegmentRows.m`) locates row boundaries.
- Task 3 (`task3SegmentCharacters.m`) locates character boundaries within rows.
- Task 4 (`task4PreprocessingCharacters.m`) resizes character images to $N \times N$ pixels.
- Task 5 (`task5CreatingAlphabet.m`) extracts and saves reference characters.
- Task 6 (`task6RecognizeCharacters.m`) matches test characters against alphabet templates.

A final wrapper script (`image_project.m`) orchestrates these six stages, reading an input document image and returning the recognized text.

2.1.1 Task 1 - Binarization

The binarization stage is responsible for converting the continuous-valued pixel intensities in a color or grayscale image into a binary (two-valued) representation where one value indicates text pixels and the other indicates background pixels. The classical approach to binarization is to apply a global threshold: select a single pixel intensity value T such that pixels with intensity less than T are classified as background, and pixels with intensity greater than T are classified as text.

2.1.1.1 Discussion: advanced methods

We examined the results of different threshold values and understood their effect on the binarized text image. The question was: how to choose T optimally? Then Otsu's method came into play. This method selects the threshold T that maximizes the between-class variance. It gives us a dynamic definition of the optimal threshold for each input image.

2.1.2 Task 2 - Row Segmentation

Once we have a binary image, the next step is to locate the text rows. To start, we applied a morphological closing to make each text line more compact. Then, we computed the number of active pixels for each row. We used, as well, a relative threshold to distinguish text rows from background rows. Finally, the start and end indices of each text line were detected and each row was extracted as a separate binary image.

2.1.2.1 Discussion: advanced methods

The morphological method of blob division wasn't tested nor implemented. This decision was based on one of the general principles in engineering: keep simpler methods unless they demonstrably fail on realistic inputs.

2.1.3 Task 3 - Character Segmentation

Character segmentation is the process of locating where individual characters begin and end within a row of text. Unlike row segmentation, which is a one-dimensional problem (finding row boundaries along the vertical axis), character segmentation is slightly more complex because characters in a row can have varying widths and can nearly touch one another. A clear example of this is the width difference between a narrow character like "I" and a wide character like "W".

The function computes a vertical projection for every extracted row image. This projection corresponds to the sum of active pixels in each column, allowing us to distinguish between regions containing characters and blank spaces.

The projection is then binarized by applying a threshold, producing a binary vector in which values equal to `1` correspond to columns containing foreground pixels, while values equal to `0` represent background regions. This additional binarization step reduces the influence of noise and facilitates the detection of character boundaries.

Character segmentation is performed by analyzing transitions in the binary projection using MATLAB's `diff()` function. Specifically:

a) `diff = 1`: indicates the beginning of a character region.

b) `diff = -1`: indicates the end of a character region.

From these transitions, the start and end positions of all character candidates are obtained.

2.1.4 Task 4 - Preprocessing

Once individual characters have been segmented, they come in many different sizes. Before we can compare characters, we must normalize them to a common size. Furthermore, we must account for aspect ratio: some characters might be wider than they are tall, others taller than wide. If we simply resize a wide, short character to a square shape

without first adjusting aspect ratio, the character will be horizontally compressed, distorting its appearance.

The function (texttttask4PreprocessingCharacters), which takes as input parameters the segmented characters from task 3 and the desired size for each image, addresses this problem.

We used cell arrays, since they allow us to store different size images (and therefore matrixes), which is what happens after character segmentation (task 3). Also, we preallocate (textttcharacterOut = cell(numchars, 1)) in order to improve efficiency and avoid redimensioning in each loop iteration.

For each array element, after computing its size, we padded with zeros because, as we know from task 1, we are working with a binary image in which characters are white (1 or true) and the background is black (0 or false). That is, we added background columns or lines to make each image square. This was done before resizing the image to avoid aspect ratio distortion (for example, without this an 'I' would increase size horizontally, looking like a white square), which would make the recognition process extremely complex and lead to many errors. Therefore, by doing it this way the character keeps its original shape.

For the actual padding process we distinguish two opposite cases:

a) The height is greater than the width.

b) The width is greater than the height

In both we calculated how much we need for the image to be squared (represented by the variable padDif), and then we divided that into right and left (or top and bottom in case the width is greater than the height). This ensures the character remains centered. Otherwise, it would be a source of error for the recognition process.

In addition, we used the Matlab functions `floor()` and `ceil()`) in case the difference between dimensions is an odd number (for example, if we have to add 7 pixels we add 3 to one side and 4 to the other, while if we used `round()`, or `ceil()` or `floor()` for both we would end up with 4 and 4 or 3 and 3, destroying the square geometry).

Finally, we resize the image to the dimension $N \times N$ passed as a parameter with the Matlab function (textttimresize()). Thus, they all have the same size and we can compare them easily.

After all the loop iterations, the resultant square images are stored in `characterOut`, the output of the function.

2.1.5 Task 5 - Alphabet creation

Before we can recognize characters, we must define what each character looks like. We do this by creating an alphabet—a collection of template images, one for each letter A through Z. The alphabet is created by processing an ideal reference document that contains one instance of each letter in the desired font and style. The process is

straightforward: we apply Tasks 1 through 4 to the reference document, segmenting it into rows, then into characters, then normalizing each character. The normalization is done with the same N to ensure that all templates share the same geometry and resolution, enabling reliable similarity comparisons during recognition. The processed templates are stored in a MATLAB structure named `alphabet`, where each field corresponds to a character label and contains its associated normalized template image. The result is a directory containing 26 template images.

2.1.6 Task 6 - Character recognition

The recognition stage is the final step of the OCR pipeline. Given a character image, we must decide which of the 26 letters it most likely represents. The approach is template matching: we compare the test character against each of the 26 alphabet templates and select the one with the highest similarity score.
The similarity metric is crucial. We use normalized correlation, computed using MATLAB's `corr2()` function, computes `(test · template) / (||test|| ||template||)`, the cosine similarity of the vectorized images. This ranges from -1 (opposite) to +1 (identical).

The recognition function iterates through all 26 alphabet templates, computing correlation with the test character for each. It then selects the character $\hat{c}$ that maximizes the correlation score:

$$\hat{c} = \arg\max_i S_i \tag{1}$$

2.1.6.1 Confidence Criterion

To improve robustness and avoid incorrect classifications, a confidence threshold is introduced. If the maximum similarity score is below a predefined threshold (empirically set to 0.5), the recognition is considered unreliable and the output is rejected:

$$\text{if } \max(S_i) < \tau \Rightarrow \hat{c} = ? \tag{2}$$

This mechanism prevents low-confidence matches from being accepted as valid predictions.

Additionally, a margin-based criterion comparing the best and second-best scores was introduced to further improve classification reliability.

2.2 On the run improvements

- Task 1: we implemented an adaptive thresholding method (Otsu's method)
- Task 3: During the initial experiments, some characters were incorrectly segmented. For instance, characters such as "M" or "N" were partially truncated due to excessively narrow segmentation boundaries. To mitigate this issue, a small margin was added to both the start and end indices of each detected segment, resulting in more complete character extractions.

- A second issue appeared when adjacent characters became connected, producing merged segments such as “TA”. To address this problem, an overlapping segmentation strategy inspired by OCR techniques was introduced. The width of each detected segment is compared with the average character width. If a segment exceeds 1.5 times the average width, it is assumed to contain multiple characters and is divided approximately at its center using MATLAB’s `round()` function. The resulting subsegments are then stored independently.